

1. OS Definition & 3 System Entities

- Operating System:** Bridge between user applications & hardware; acts as **Resource Allocator** (fairness) & **Control Program** (prevents errors).
- 3 System Entities:**
 - Hardware:** Bare CPU, ALU, registers, RAM, I/O devices.
 - System Software:** OS Kernel, device drivers, translators.
 - Application Software:** User-facing programs (Browsers, IDEs).
- Translators:**
 - Assembler:** Assembly mnemonics → Machine code (0/1).
 - Compiler:** Entire HLL (C/C++) code parsed together into object code.
 - Interpreter:** Line-by-line interpretation & execution (Python/JS).

2. User View vs System View

- User View:** Primary goal = **Ease & Convenience**. Fast response time, seamless UI, zero hardware interaction needed.
- System View:** Primary goal = **Resource Efficiency & Fairness**. CPU scheduling, memory partitioning, conflict resolution.
- Execution Trace ($a = b + c$):**
 - High-level C code parsed by compiler.
 - Assembly generated: `MOV EAX, [b]`, `ADD EAX, [c]`, `MOV [a], EAX`.
 - ALU executes addition; MMU updates physical RAM byte address.

3. Computer System Organization & Memory Hierarchy

CPU (ALU+CU)

↔ Common Bus ↔

Memory Controller (RAM)

↔ Common Bus ↔

Device Controllers (Disks/USB)

- Device Controller & Buffer:** Har peripheral controller ke paas local hardware buffer hota hai. Data transfer complete hone par hardware interrupt trigger karta hai.
- Memory Pyramid:** Registers (<1ns) → L1/L2/L3 Caches (1-10ns) → Main Memory RAM (50-100ns) → SSD/HDD Disk (>1ms).
- AMAT Formula:** $AMAT = Hit\ Time + (Miss\ Rate \times Miss\ Penalty)$.
- Cache Writes:** *Write-Through* (RAM updated instantly) vs *Write-Back* (Dirty bit track; RAM updated only on block replacement).

4. Dual-Mode Operation (Hardware Security Foundation)

Prevents rogue/buggy user processes from overwriting OS memory or executing illegal hardware commands.

User Mode (Mode Bit = 1, Ring 3)

- Unprivileged instructions only.
- Cannot modify interrupt vector, base/limit regs, or direct I/O.

Kernel Mode (Mode Bit = 0, Ring 0)

- Privileged instructions allowed.
- Full hardware access to all CPU registers & physical RAM.

App: `printf("Hello")` → Trap / syscall → **Mode Bit: 1 → 0** → `sys_write()` in Kernel → **Mode Bit: 0 → 1** → Returns to App

5. Bootstrapping Program & Cold vs Warm Booting

- Bootstrapping Sequence:** Power ON → Power Good signal → CPU PC loads fixed address in ROM BIOS/UEFI → **POST (Power-On Self-Test)** probes hardware → BIOS reads MBR/GPT sector from Disk → Bootloader loads Kernel image into RAM → Initializes system daemons.

Comparison Parameter	Cold Booting (Hard Boot)	Warm Booting (Soft Boot / Reboot)
Trigger Mechanism	Power button ON from complete 0V OFF state	OS restart option, <code>Ctrl+Alt+Del</code> , <code>sudo reboot</code>
POST Diagnostic	Full hardware self-test diagnostic executed	POST is completely bypassed (Hardware powered)
Boot Duration	Significantly slower due to POST checks	Much faster; clears hung memory state & reloads kernel

1. Operating System Types: Comprehensive Comparison Matrix

OS Type	Working Mechanism	Primary Advantage	Key Limitation / Trade-off
Batch OS	Punch cards grouped by human operator into batches (FORTRAN, COBOL); sequential run.	Zero setup time between identical jobs; automated monitor.	Zero interactivity; CPU completely idle during I/O wait.
Multiprogrammed	Multiple processes simultaneously resident in RAM. Non-preemptive switch on I/O.	Maximizes CPU utilization (CPU idle time drops from 70% to <10%).	Requires complex memory allocation & process scheduling.
Time-Sharing	Logical extension of multiprogramming; Round Robin with Time Quantum (q); Preemptive.	Minimizes response time; interactive feel across multiple users.	Context switch latency overhead if quantum q is too small.
Hard RTOS	Strict deterministic deadlines ($WCET \leq D$). Miss deadline = Catastrophic failure.	Guaranteed timing; zero jitter; predictable interrupt latency.	Virtual memory/paging disabled (to eliminate page fault latency).
Soft RTOS	Prioritized deadlines. Miss deadline = Graceful QoS degradation (lag/frame drop).	Supports high-throughput multimedia & dynamic caching.	Unsuitable for safety-critical missions (pacemakers/avionics).
SMP Multiprocessor	All CPUs equal peers sharing common bus, clock, and RAM. Dynamic load balancing.	High throughput; graceful degradation (fault-soft reliability).	Contention on shared system memory bus and cache coherency.
ASMP Multiprocessor	Master-Slave: Master CPU controls OS & I/O; Slave CPUs execute user tasks.	Simple design; no race conditions in kernel data structures.	Master CPU is a major bottleneck; Master failure crashes system.

2. Spooling vs Buffering

- Buffering:** Small volatile RAM area jo single device aur CPU ke beech speed mismatch ko temporarily absorb karta hai.
- Spooling (Simultaneous Peripheral Operation On-Line):** High-speed hard disk par dedicated FIFO queue buffer. CPU directly disk par print jobs push kar deta hai; slow physical printer disk spool se independent print karta rehta hai.

3. Heavyweight Process vs Thread

- Process:** Program in execution with isolated address space (Code, Data, Heap, Stack). Heavy context switch (TLB flush).
- Thread (LWP):** Lightweight unit inside process.
 - Shared: Code, Data, Heap, Open Files.
 - Private: Program Counter (PC), Registers, Stack.

4. High-Yield Mathematical Thought Process & Formulas

CPU Utilization = $1 - p^n$

- p = Process I/O wait probability (e.g. 70% = 0.70)
- n = Degree of Multiprogramming (Jobs in RAM)
- Thought Process:**
 - Uniprogramming ($n=1$): Util = $1 - 0.70 = 30\%$
 - Multiprogramming ($n=4$): Util = $1 - (0.70)^4 = 76\%$
 - Multiprogramming ($n=8$): Util = $1 - (0.70)^8 = 94.2\%$

Amdahl's Speedup $S = 1 / [(1 - P) + P/N]$

- P = Parallel fraction, N = Number of cores
- $(1 - P)$ = Strictly serial code fraction
- Thought Process:**
 - If serial fraction is 20% ($1-P = 0.20$):
 - Max theoretical speedup even with infinite cores ($N \rightarrow \infty$) is strictly capped at: $1 / 0.20 = 5.0\times!$

Time-Sharing Context Switch Overhead = $\delta / (q + \delta)$ | Optimal Rule: 80% of CPU bursts $< q$

1. OS Architecture Approaches: Monolithic vs Layered vs Microkernel vs Modular

Architecture	Subsystem Placement	Communication	Reliability & Fault Tolerance	Performance
Simple (MS-DOS)	No separation; apps access BIOS directly.	Direct jumps	Very Poor (App bug corrupts BIOS)	Fastest (No protection)
Monolithic (Linux Core)	All services (VMM, scheduler, drivers) in Kernel Ring 0.	Direct C function calls	Poor (Third-party driver bug crashes entire kernel)	Fastest (Zero IPC overhead)
Layered (THE System)	N tiered layers; Layer i only calls Layer i-1.	Strict vertical calls	High modularity; easy layer-by-layer debugging.	Slower (Call stack traversal)
Microkernel (Mach, QNX)	Minimal Ring 0 (IPC, scheduling). Drivers/FS in User Mode.	IPC Message Passing	Extreme (Driver crash restarts isolated user daemon)	Slower (IPC context switch delay)
Modular LKM (Linux LKMs)	Core kernel loads dynamic <code>.ko</code> modules at runtime.	Direct C function calls	High (Modules load/unload on-the-fly without reboot)	Fast (Monolithic speed + modularity)

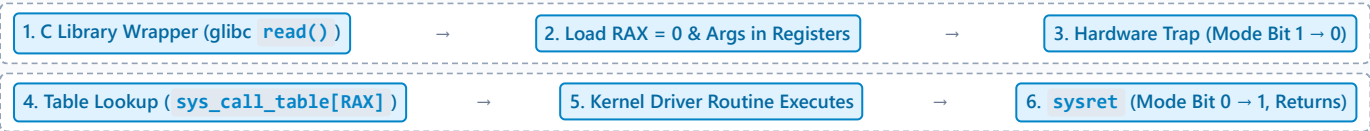
2. Reentrant Kernel Concurrency

- **Definition:** Permits multiple processes to execute concurrently inside kernel mode without encountering data inconsistencies.
- **Mechanism:**
 - Har process ka independent **Kernel Stack** hota hai.
 - Shared kernel structures are guarded via fine-grained **Spinlocks & Mutexes**.
 - Eliminates global interrupt disabling of ancient systems.

3. Parameter Passing in Syscalls

- **1. CPU Registers:** Fastest (RDI, RSI, RDX in Linux x86_64 ABI); limited to ≤ 6 args.
- **2. Memory Block / Table:** Parameters packed in struct; address pointer passed in register.
- **3. Program Stack:** Parameters pushed onto user stack; kernel pops them off.

4. 6-Stage System Call Execution Lifecycle (Step-by-Step Hardware & Software Trace)



5. The 8 Core Operating System Subsystems (Slide 135 Mind Map)

- **Process Management:** Creation, PCB, CPU scheduling, IPC, Deadlocks.
- **Main Memory:** Allocation byte tracking, Virtual Memory, Paging, Base/Limit.
- **File Management:** Inodes, Directory tree, file CRUD, disk block mapping.
- **I/O System:** Buffer caching, print spooling, uniform device driver API.
- **Secondary Storage:** Free space bitmaps, disk head arm scheduling (SCAN/SSTF).
- **Networking:** Distributed routing, TCP/IP stack, socket abstraction.
- **Protection & Security:** Dual-mode enforcement, ACLs, memory isolation.
- **Command Interpreter:** CLI Shell / GUI parsing user input into system calls.